

Contents

Steam Reviews — Module 3 Lab Guide	2
The Scenario — Your Mission	2
Where you work	2
The problem	2
Your manager's request	3
Your team's job for the next 2 weeks (Module 3)	3
After Module 3	3
How This Guide Works (Read This First!)	3
We do NOT give you the full code	3
Visualizations — Two Modes	4
1. Exploratory charts (for YOU)	4
2. Explanatory charts (for ERIK)	4
Your plotting toolkit (you learned this in Module 2 Class 5)	4
Before You Start — Google Colab Setup	5
Step 1 — Open a new Colab notebook	5
Step 2 — Connect to Google Drive	5
Step 3 — Create a project folder	5
Step 4 — Get the data	5
Step 5 — Test it (read just a few rows)	6
Colab tips	6
Class 1 — Data Cleaning	6
Your goal	6
Inputs	6
Outputs	6
Phase A — Explore the data first (15 minutes)	6
Phase B — Clean the reviews table (45 minutes)	7
Phase C — Make ONE chart for Erik (15 minutes)	9
Common mistakes in Class 1	10
Self-check before Class 2	10
Class 2 — Encoding and Scaling	10
Your goal	10
Inputs	10
Outputs	11
Phase A — Explore the data first	11
Phase B — Encode and scale	11
Phase C — Make ONE chart for Erik	13
Common mistakes in Class 2	13
Self-check before Class 3	13
Class 3 — Feature Engineering	14
Your goal	14
Inputs	14
Outputs	14
Phase A — Explore the data first	14
Phase B — Engineer the features	15
Phase C — Make ONE chart for Erik	17
Common mistakes in Class 3	17
Self-check before Class 4	17
Class 4 — Feature Selection	18

Your goal	18
Inputs	18
Outputs	18
Phase A — Explore the data first	18
Phase B — Select features	19
Phase C — Make ONE chart for Erik	20
Common mistakes in Class 4	20
Self-check before Class 5	20
Class 5 — Pipelines	21
Your goal	21
Inputs	21
Outputs	21
Phase A — Explore the data first	21
Phase B — Build the pipeline	22
Phase C — Make ONE chart for Erik	23
Common mistakes in Class 5	23
Self-check before Class 6	23
Class 6 — End-to-End Lab (THE BIG ONE)	24
Your goal	24
Time	24
Inputs	24
Outputs	24
Phase A — Explore the data first (10 minutes)	24
Phase B — Build the final dataset (70 minutes)	24
Phase C — Findings report for Erik (10 minutes)	25
Grading rubric (100 points)	26
Submit	26
Tips for the whole module	26

Steam Reviews — Module 3 Lab Guide

Scenario: Steam Game Reviews. Predict if a player recommends a game. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at **Valve**. Valve owns **Steam**. Steam is the biggest store in the world for video games on a computer. More than 100 million people use Steam every month.

Every day on Steam: - People **buy games**. - People **play games for many hours**. - After playing, people **write a review** of the game. - At the end of each review, the player clicks one of two big buttons: - “**Recommended**” (thumbs up) - “**Not recommended**” (thumbs down)

Steam has **more than 6 million reviews** in our dataset. That is a huge number. About **80% of all reviews are “Recommended”**. Only 20% are “Not recommended”.

The problem

When a shopper opens a game page on Steam, they see two numbers: the count of recommended and not-recommended reviews. Then they see a long list of review texts. The shopper has 30 seconds. Which reviews

should we show first?

Right now, Steam shows the **most “helpful” reviews** at the top. But many of those are funny jokes. Some are very short (“10/10 buy this game”). The shopper does not learn anything new from those reviews.

Our team thinks the **best reviews are smart “Not recommended” reviews**. Why? - A long, careful “Not recommended” review tells the shopper the real problems. - A long, careful “Not recommended” review SAVES the shopper from a bad purchase. - A long, careful “Not recommended” review makes the shopper trust Steam.

But we cannot read 6 million reviews by hand. We need a model that: 1. Reads the **review text**. 2. Predicts: would this person click “Recommended” or “Not recommended”? 3. Helps us find the **smart critics** — people whose words match their thumb.

Your manager’s request

Your manager, **Erik** (Steam Reviews Product Lead at Valve), tells you in the team meeting:

“Every game has thousands of reviews. Most are ‘recommended’ (thumbs up). But a ‘NOT recommended’ review is more valuable — it warns a shopper. We want to find the smart critics.

Build me a model. **Predict the recommend label from the text**. Then surface the BEST not-recommended reviews — the long, careful ones, not the angry one-liners.

If we show shoppers the 3 best critical reviews on every game page, we will save them from bad purchases. Steam will become the most trusted store on the internet.”

Your team’s job for the next 2 weeks (Module 3)

Erik cannot do this alone. Our data has **6.4 million reviews**. That is too big to open in Excel. The file has **8 columns**, mixed languages, and very long tails (some players have 10,000+ hours played).

Your job in Module 3: > **Turn a huge 6.4M-row file into ONE clean file of about 100,000 rows. Filter to English. Build smart new columns. The clean file will be used to train the model in Module 4.**

The clean file is called `steam_reviews_clean.parquet`. It must have **about 19 specific columns** (we will see them in Class 6).

After Module 3

Module	What happens
Module 4	Train the prediction model. Erik finally gets his “recommend score” from text.
Module 5	Find groups of reviewers (super-fans vs hard critics vs new players).
Module 7	Read the review text. Find common complaints. Find the smart critics.

You use the **same Steam Reviews dataset** until the end of Module 7.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see two kinds of content:

1. WHAT / WHY / EXPECTED — explained in words

For every step you read: - **WHAT** you have to do (in normal language) - **WHY** you do it (the reason) - **HINTS** — the function names, the arguments to remember - **EXPECTED** — the result you should see if you did it right

2. Sometimes a tiny code skeleton with blanks

For the harder steps we give you a code skeleton with ____ blanks. **You fill in the blanks.**

Why no full code?

Because you will NOT learn if you copy-paste. - In Module 4, the model only works if YOU understand each step. - In your future job, nobody will give you the code. - Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

Rule: If you finish a step in 30 seconds by copy-pasting from a teammate's notebook, do it AGAIN by yourself. Different variable names, same logic.

Visualizations — Two Modes

In every class you will make charts. There are **two reasons** to make a chart:

1. Exploratory charts (for YOU)

Made BEFORE you clean the data. To understand what the data looks like. - Fast, ugly, no labels needed. - Examples: `df.hist()`, `df['column'].value_counts().plot.bar()`.

Goal: “**What does this data look like?**”

2. Explanatory charts (for ERIK)

Made AFTER you understand. To EXPLAIN something to your manager. - Clean, labeled, one clear message. - Must have: title, x-label, y-label, source, and a 1-sentence takeaway.

Goal: “**Erik, look at this. This is the pattern.**”

In every class you make BOTH kinds.

Your plotting toolkit (you learned this in Module 2 Class 5)

Plot	When to use it	Function name
Histogram	See the SHAPE of a numeric column	<code>plt.hist()</code> or <code>sns.histplot()</code>
Bar chart	Count of each category	<code>df['col'].value_counts().plot.bar()</code>
Box plot	See outliers in a numeric column	<code>sns.boxplot()</code>
Scatter plot	See the relationship between 2 numbers	<code>plt.scatter()</code> or <code>sns.scatterplot()</code>
Heatmap	See correlation between many columns	<code>sns.heatmap(df.corr())</code>
Line plot	See change over time	<code>plt.plot()</code>

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

You will work in **Google Colab**. That means: - You do NOT install Python. - You do NOT install pandas, scikit-learn, or anything. - You just open a notebook in your web browser.

Step 1 — Open a new Colab notebook

1. Go to <https://colab.research.google.com>
2. Sign in with your Google account.
3. Click “**New notebook**” (top left).
4. Name it `steam_module_3.ipynb`.

Step 2 — Connect to Google Drive

Colab files **disappear** when you close the browser. So you must save your work to **Google Drive**.

In the first cell:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up asks for permission. Click “Allow”. After this, Drive is available at `/content/drive/MyDrive/`.

Step 3 — Create a project folder

In a new cell:

```
import os
os.makedirs('/content/drive/MyDrive/steam_lab', exist_ok=True)
%cd /content/drive/MyDrive/steam_lab
```

Your team will save ALL files in this folder.

Step 4 — Get the data

WARNING: The full file is **about 6.4 million rows**. The download is large (~3 GB unzipped). You **MUST** sample it down BEFORE you do anything else. We will do this in Class 1.

Option A — Direct from Kaggle (recommended):

1. Make a free Kaggle account.
2. Go to kaggle.com/settings/account, scroll to “API”, click “Create New Token”. Save the `kaggle.json` file.
3. In Colab:

```
from google.colab import files
files.upload()    # pick kaggle.json
```

4. Then:

```
!mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
!pip install kaggle -q
!kaggle datasets download -d andrewmvd/steam-reviews
!unzip -q steam-reviews.zip -d data
!ls data/
```

You should see **1 large CSV file** inside `data/` (around 3 GB).

Option B — Upload by hand:

1. Download the zip from kaggle.com/datasets/andrewmvd/steam-reviews on your laptop.
2. Unzip it.
3. In Colab’s file panel (left sidebar), upload the CSV file. (This is slow — Option A is faster.)

Step 5 — Test it (read just a few rows)

Do NOT load the full file. It will eat all your memory.

```
import pandas as pd
preview = pd.read_csv('data/dataset.csv', nrows=1000)
print(preview.shape)
print(preview.columns.tolist())
```

Should print something like (1000, 8) and a list of 8 column names. You are ready.

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate .parquet files to Drive	Files in /content/ disappear when Colab disconnects.
Share the notebook with teammates (Share button, top right)	Editor access. Like Google Docs for code.
Restart runtime if memory full	Runtime > Restart runtime.
Use Runtime > Change runtime type > High-RAM if you can	This dataset is big.

Class 1 — Data Cleaning

Scenario reminder: Erik drops one HUGE CSV file on your desk. 6.4 million rows. Many languages. Crazy long-tail numbers. Your job today: cut it down to 100,000 English rows, fix the basic columns, and save a small clean file.

Your goal

Make the giant file USABLE. Filter to English. Sample 100,000 rows. Convert Unix timestamps. Find missing values.

Inputs

- The 1 large CSV file in data/ (~6.4M rows)

Outputs

- steam_step1.parquet saved in your Drive folder (~100,000 rows)
- 3+ exploratory charts in your notebook
- 1 explanatory chart for Erik
- Notes in markdown about every decision you made

Phase A — Explore the data first (15 minutes)

Before you clean anything, **LOOK** at the data. But do NOT load the full file. Use `nrows` to read only a small piece.

Exploratory chart 1 — Language distribution (the most important chart today)

- **Question:** “Of all the reviews, how many are in English? How many in Russian, Chinese, German?”
- **HINTS:**
 - Read maybe 500,000 rows for this check: `pd.read_csv('data/dataset.csv', nrows=500000)`.
 - Use `sample['language'].value_counts().head(15)`.
 - Then `.plot.bar()` on the result.
- **What you learn:** English is the biggest group, but not the only one. There are many languages. We **MUST** filter to English before sampling.

Exploratory chart 2 — How many reviews are “recommended”?

- **Question:** “Of the 500,000 preview rows, how many have `recommended = True`?”
- **HINTS:**
 - Use `sample['recommended'].value_counts(normalize=True)`.
 - Then `.plot.bar()`.
- **What you learn:** The target is **unbalanced**. About 80% recommended, 20% not recommended. This will matter in Class 5.

Exploratory chart 3 — Missing values per column

- **Question:** “Which columns have the most missing data?”
 - **HINTS:**
 - Use `sample.isna().sum().sort_values(ascending=False)`.
 - Plot it as a bar chart.
 - **What you learn:** Some columns are almost always missing. Some are always filled. The text review column has some empty values.
-

Phase B — Clean the reviews table (45 minutes)

Step 1 — Read the file in chunks AND filter to English AT THE SAME TIME

- **WHAT:** The file is too big to load. We will read it in pieces of 500,000 rows. From each piece, keep **ONLY** the English rows. Add the small piece to a list. At the end, join all the pieces.
- **HINTS:**
 - Use `pd.read_csv('data/dataset.csv', chunksize=500000)`. This gives you an iterator.
 - Loop: `for chunk in iterator: then english_chunk = chunk[chunk['language'] == 'english']`.
 - Append `english_chunk` to a Python list.
 - After the loop: `df_english = pd.concat(your_list, ignore_index=True)`.
- **WHY:** If you load 6.4M rows into memory and then filter, Colab crashes. Filter as you go.
- **EXPECTED:** About **2-3 million English rows** after the loop (English is the biggest language but not all of the data).

Step 2 — Sample 100,000 rows with a fixed seed

- **WHAT:** Even 2 million English rows is too many for a lab. Take a random 100,000.
- **HINTS:**
 - Use `df_english.sample(n=100000, random_state=___)`.
 - Fill in the blank with the seed 42. **Always 42.** Otherwise your teammates get different numbers than you.
- **WHY random_state=42?** Reproducibility. Two students with the same code get the same rows.
- **EXPECTED:** A DataFrame `df` with exactly 100,000 rows.

Step 3 — Look at each column

- **WHAT:** Check `.shape`, `.info()`, and `.head()` of `df`.

- **HINTS:**
 - Look at the Dtype column in `.info()` output.
 - **Is `timestamp_created` a number (Unix time) or a date?** Right now it is a big integer.
 - Is recommended bool or text “True”/“False”?
- **EXPECTED:**

Column	What it is	Dtype
<code>app_id</code>	game id	int
<code>app_name</code>	game title	object (string)
<code>review_id</code>	unique id of the review	int
<code>language</code>	language of the review	object
<code>review</code>	the text of the review	object
<code>timestamp_created</code>	when the review was written, Unix seconds	int
<code>timestamp_updated</code>	last edit	int
<code>recommended</code>	True / False	bool
<code>votes_up</code>	how many users marked this review as helpful	int
<code>votes_funny</code>	how many users marked this review as funny	int
<code>weighted_vote_score</code>	Steam’s own helpfulness score, 0 to 1	float
<code>comment_count</code>	comments on the review	int
<code>steam_purchase</code>	was the game bought on Steam?	bool
<code>received_for_free</code>	did the player get the game for free?	bool
<code>written_during_early_access</code>	was the review in early access?	bool
<code>author.steamid</code>	the writer’s id	int
<code>author.num_games_owned</code>	how many games the writer owns	int
<code>author.num_reviews</code>	how many reviews the writer has written	int
<code>author.playtime_forever</code>	total hours played, in MINUTES	float
<code>author.playtime_last_two_weeks</code>	recent hours, in minutes	float
<code>author.playtime_at_review</code>	hours played when the review was written, MINUTES	float
<code>author.last_played</code>	Unix time of the last play	float

NOTE: The exact column names depend on the dataset version. Use `df.columns.tolist()` to see the real names. Some are `author.playtime_forever`, some are simpler.

Step 4 — Convert `timestamp_created` to a real date

- **WHAT:** `timestamp_created` is a big integer like 1483228800. That is **Unix time** (seconds since 1 January 1970). Turn it into a real date.
- **HINTS:**
 - Use `pd.to_datetime(df['timestamp_created'], unit='s')`.
 - The argument `unit='s'` tells pandas the number is SECONDS, not nanoseconds.
 - Save into a new column: `df['review_date'] = ____`.
- **WHY:** If `timestamp_created` is a giant integer, you cannot say “reviews from 2019”. You cannot use `.dt.year`. You must convert.
- **EXPECTED:** `df['review_date'].dtype` is `datetime64[ns]`. Run `df['review_date'].min()` and `.max()` — should be between 2010 and 2020.

Step 5 — Convert `author.playtime_forever` from minutes to HOURS

- **WHAT:** Steam stores playtime in **minutes**. A player with 6,000 minutes has played 100 hours. Convert to hours.

- **HINTS:**
 - `df['hours_played'] = df['author.playtime_forever'] / 60.`
 - The column name might be slightly different — check with `df.columns.tolist()`.
- **EXPECTED:** `df['hours_played']` has values from 0 to 10,000+. The mean is around 100. The median is around 15.

Step 6 — Find missing values

- **WHAT:** Count missing values per column.
- **HINTS:** `.isna()` returns True/False per cell. `.sum()` counts the Trues per column.
- **EXPECTED:** Something like:

review	54
author.playtime_at_review	1230
weighted_vote_score	0

Step 7 — Drop rows where review is missing

- **WHAT:** If the review text is empty, we cannot use it for Module 7. Drop those rows.
- **HINTS:**
 - Use `.dropna(subset=['review'])`.
- **WHY:** The whole project is about predicting from text. No text = no project.
- **EXPECTED:** A few hundred rows removed.

Step 8 — Drop rows where the review is just spaces

- **WHAT:** Some reviews are not “missing” but they are an empty string "" or just spaces.
- **HINTS:**
 - Filter: `df = df[df['review'].str.strip() != '']`.
 - `.str.strip()` removes spaces at the start and end.
- **EXPECTED:** A few more rows removed.

Step 9 — Write down what you did

In a markdown cell, write: - Starting rows: ~6,400,000 - After English filter: ~2,500,000 - After random sample (seed 42): 100,000 - After drop missing review: ~99,940 - After drop blank reviews: ~99,900 - WHY you removed each group.

Step 10 — Save to Drive

- **WHAT:** Save the cleaned DataFrame as parquet, inside your Drive folder.
- **HINTS:** `df.to_parquet('/content/drive/MyDrive/steam_lab/steam_step1.parquet')`.
- **WHY parquet, not CSV?** Parquet is 10x smaller and 10x faster than CSV for big data. Also it remembers the dtypes (so dates stay as dates).

Phase C — Make ONE chart for Erik (15 minutes)

He does not have time to read your code. He wants ONE picture.

Erik’s chart — “Language distribution before filtering”

A bar chart showing the top 10 languages in the full data, BEFORE we filter to English.

- **HINTS:**

- Reuse the count from the preview (Phase A chart 1) or count from the chunks.
- Use `plt.bar()` with 10 bars.
- Add the title: "Steam reviews by language – English is 40% of all reviews".
- X-label: language.
- Y-label: number of reviews (or %).
- Add a note: "We will work only with English for this project."
- **Takeaway for Erik:** "English is the largest group. But 60% of all Steam users review in other languages. In a future project we should not ignore them."

Common mistakes in Class 1

Mistake	What goes wrong
Try to load the full 6.4M-row file at once Sample BEFORE filtering to English Forget <code>random_state=42</code>	Colab crashes with "out of memory". You get a mixed sample. Only ~40% will be English. Each teammate gets different rows. Numbers do not match.
Forget <code>unit='s'</code> on <code>to_datetime</code>	Pandas thinks the integer is nanoseconds. Dates jump to year 51000.
Save to <code>/content/</code> instead of Drive Keep <code>playtime</code> in minutes	File disappears when Colab disconnects. Charts show 6,000,000 minutes. Erik thinks something is broken.

Self-check before Class 2

- ☐ You read the file in chunks (not all at once).
- ☐ You filtered to `language == 'english'` BEFORE sampling.
- ☐ You sampled 100,000 rows with `random_state=42`.
- ☐ `review_date` is a real datetime column.
- ☐ `hours_played` is in HOURS, not minutes.
- ☐ At least 3 exploratory charts in your notebook.
- ☐ 1 polished chart for Erik (language distribution).
- ☐ You saved `steam_step1.parquet` to your Drive folder.
- ☐ You wrote down (in markdown) what you did and WHY.

Class 2 — Encoding and Scaling

Scenario reminder: Erik looks at your cleaned data. He is happy. But he says: "The numeric columns look strange. One player has 10,000 hours played. Most have 5. The model will be confused. Fix the scales. Also, recommended is True/False. The model needs a number."

Your goal

Turn TEXT and BOOLEAN columns into numbers. Apply log-transform to the long-tail numeric columns. Make all numeric columns about the same size.

Inputs

- `steam_step1.parquet` from Class 1

Outputs

- steam_step2.parquet in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Erik
-

Phase A — Explore the data first

Exploratory chart 1 — Distribution of hours_played (RAW)

- **Question:** “What is the shape of hours played? Is it long-tailed?”
- **HINTS:**
 - Use a histogram (`plt.hist()`).
 - Use `bins=100`.
 - Try `plt.xlim(0, 500)` to zoom in on most of the data.
- **What you learn:** The shape is HEAVY long tail. Most players have 0-20 hours. A few have 5,000+ hours. The model cannot handle this raw.

Exploratory chart 2 — Distribution of hours_played AFTER `np.log1p`

- **Question:** “Does log-transform make the shape more normal?”
- **HINTS:**
 - Make a NEW column: `df['hours_played_log'] = np.log1p(df['hours_played'])`.
 - Histogram it. Use `bins=50`.
 - Compare to chart 1.
- **What you learn:** Log makes the long tail manageable for the model. The shape now looks more like a bell.

Exploratory chart 3 — Hours played vs recommend rate (the Erik chart for THIS class)

- **Question:** “Do players with MORE hours played more often click ‘recommended’?”
- **HINTS:**
 - Bin hours_played into 6 buckets: 0-1, 1-5, 5-20, 20-100, 100-500, 500+.
 - Use `pd.cut(df['hours_played'], bins=[0, 1, 5, 20, 100, 500, 100000])`.
 - GroupBy the bin, compute `mean()` of recommended.
 - Multiply by 100 for percent.
 - Bar chart.
- **What you learn:** Players with more hours are MORE likely to click “recommended”. This is intuitive (if I hate a game, I stop playing). But it is also a useful pattern for the model.

Exploratory chart 4 — votes_up distribution

- **Question:** “How helpful are most reviews?”
 - **HINTS:** Histogram. Almost all reviews have 0 or 1 votes. A few have 1,000+.
 - **What you learn:** This column also has a long tail. We will also log-transform it.
-

Phase B — Encode and scale

Step 1 — Load steam_step1.parquet

- **WHAT:** Read your file back into memory.
- **HINTS:** `df = pd.read_parquet('/content/drive/MyDrive/steam_lab/steam_step1.parquet')`.
- **EXPECTED:** ~99,900 rows.

Step 2 — Convert recommended from boolean to 0/1

- **WHAT:** Right now recommended is True / False. Turn it into 1 / 0.
- **HINTS:**
 - Use `.astype(int)`.
 - Save into the same column or a new one: `df['recommended'] = df['recommended'].astype(int)`.
- **WHY:** Math models do not understand True and False. They need numbers.
- **EXPECTED:** `df['recommended'].mean()` is about 0.80 (80% recommended).

Step 3 — Same for the other boolean columns

- **WHAT:** Convert `steam_purchase`, `received_for_free`, `written_during_early_access` to 0/1.
- **HINTS:** Same as Step 2. You can write a for loop over the column names.

Step 4 — Log-transform the long-tail numeric columns

Original	New column
hours_played	hours_played_log
votes_up	votes_up_log
votes_funny	votes_funny_log
author.num_games_owned	num_games_owned_log
author.num_reviews	num_reviews_log

- **HINTS:**
 - `df['new_col'] = np.log1p(df['old_col'])`.
 - Use `np.log1p`, NOT `np.log`. The “1p” means “1 plus”. So `log1p(0)` is `log(1)` which is 0. Safe for zeros.
- **WHY?** `log(0)` is minus infinity, which crashes the model. `log1p(0)` is just 0.
- **WHY long-tail in general?** A player with 10,000 hours is not 10,000x more important than a player with 1 hour. The model thinks linearly. Log compresses the tail.

Step 5 — Look at categorical columns

- **WHAT:** Which columns are STRINGS that we might want to encode?
- **THINK:**
 - `app_name`: the game title. There are thousands of games. Too many to one-hot.
 - `language`: we already filtered to English, so this column is constant. **Drop it.**
- **DECIDE:**
 - For `app_id` (the game id) — we keep it for grouping, but we do NOT one-hot encode it. There are too many games.
 - Optional: we will use `app_id` later as a “target-encoded” feature in Class 3 (replace each game with its average recommend rate).

Step 6 — Drop columns we no longer need

- **WHAT:** Drop `language` (always “english” now). Drop `timestamp_updated` (we keep `timestamp_created`). Drop `comment_count` if it is mostly zero.
- **HINTS:**
 - `df = df.drop(columns=['language', 'timestamp_updated'])`.
- **WHY:** Fewer columns = simpler model = faster training.

Step 7 — Scale numeric columns (preview only)

- **WHAT:** Use `StandardScaler` to make every numeric column have mean = 0, std = 1.
- **HINTS:**

- from sklearn.preprocessing import StandardScaler.
- scaler = StandardScaler().
- scaled = scaler.fit_transform(df[numeric_cols]).
- **WARNING:** This is for inspection only. In Class 5, scaling goes INSIDE the Pipeline. Do NOT save the scaled version as your final file.

Step 8 — Save

- **WHAT:** Save to Drive. `df.to_parquet('/content/drive/MyDrive/steam_lab/steam_step2.parquet')`.
- **Save the UNSCALED version.** Scaling will happen inside the Pipeline later.

Phase C — Make ONE chart for Erik

Erik's chart — “Hours played vs recommend rate”

Use the Phase A chart 3 (hours played bucket vs recommend rate). Polish it for Erik.

- **HINTS:**
 - 6 bars: one per hours bucket.
 - Y-axis: % recommended.
 - Put the % on top of each bar.
- **Title:** “Players with more hours played are more likely to recommend - 95% at 500+ hours vs 60% at 0-1 hours”.
- **X-label:** Hours played bucket.
- **Y-label:** % recommended.
- **Takeaway for Erik:** “If a player keeps playing for 500+ hours, they almost always recommend. The valuable critical reviews come from players with 1-20 hours.”

Common mistakes in Class 2

Mistake	What goes wrong
Forget <code>np.log1p</code> and use <code>np.log</code> on zeros	Crashes. <code>np.log(0) = -inf</code> .
Forget <code>.astype(int)</code> on recommended	Model crashes on <code>bool</code> dtype later.
Keep language column after filtering	Column is constant. Adds noise, no signal.
Scale BEFORE train/test split	Leakage in Module 4.
Save the scaled version	Class 5 will scale again inside the pipeline. Double scaling = wrong numbers.
One-hot encode <code>app_id</code>	Thousands of games. Table explodes to 10,000 columns.

Self-check before Class 3

- ☐ recommended is int 0/1.
- ☐ Boolean columns are int 0/1.
- ☐ `hours_played_log`, `votes_up_log`, `votes_funny_log` exist.
- ☐ language column is dropped.
- ☐ At least 3 exploratory charts.
- ☐ 1 chart for Erik.
- ☐ `steam_step2.parquet` saved to Drive.

Class 3 — Feature Engineering

Scenario reminder: Erik says: “The original columns are OK, but the TRULY useful columns are not there. For example, how LONG is the review? Does the player use a lot of CAPITAL letters (shouting)? Does the review use emoji? You must MAKE these new columns.”

Your goal

Make NEW columns from the existing ones. Date features. Text features. Helpfulness features. These will help the model predict recommended.

Inputs

- steam_step2.parquet

Outputs

- steam_step3.parquet in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Erik
-

Phase A — Explore the data first

Exploratory chart 1 — Review length distribution

- **Question:** “How long are Steam reviews? Are most short (1 word) or long (1000 words)?”
- **HINTS:**
 - Make a quick column: `df['review_length'] = df['review'].str.len()`.
 - Histogram with `bins=100`.
 - Use `plt.xlim(0, 2000)` to zoom in.
- **What you learn:** Most reviews are short (10-200 characters). A few are very long (5000+). Long tail. We will log-transform later.

Exploratory chart 2 — Review length vs recommend rate

- **Question:** “Are long reviews more ‘recommended’ or more ‘not recommended’?”
- **HINTS:**
 - Bin `review_length` into 5 buckets: 0-50, 50-200, 200-500, 500-2000, 2000+.
 - `pd.cut(df['review_length'], bins=[0, 50, 200, 500, 2000, 50000])`.
 - GroupBy the bucket, mean of recommended.
- **What you learn:** Short reviews are often emotional (“buy this game” or “garbage”). Long reviews are more often critical. This is exactly what Erik wants to find.

Exploratory chart 3 — Reviews per year

- **Question:** “When were the reviews written?”
- **HINTS:**
 - Make `df['review_year'] = df['review_date'].dt.year`.
 - `df['review_year'].value_counts().sort_index().plot.bar()`.
- **What you learn:** Steam has been growing. Most reviews are from 2017-2019.

Exploratory chart 4 — weighted_vote_score vs recommended

- **Question:** “Do helpful reviews (high score) tend to be recommended or not recommended?”
- **HINTS:**
 - Box plot: `sns.boxplot(x='recommended', y='weighted_vote_score', data=df)`.
- **What you learn:** Helpfulness alone does not predict recommend. There are smart critics with high scores.

Phase B — Engineer the features

Step 1 — Date-derived features

Make these new columns from `review_date`:

New column	What it is
<code>review_year</code>	The year
<code>review_month</code>	The month (1-12)
<code>review_dayofweek</code>	0=Monday, 6=Sunday

- **HINTS:**
 - Use the `.dt` accessor.
 - `df['col'].dt.year, .dt.month, .dt.dayofweek`.
- **WHY:** A model can learn “weekend reviews are angrier” only if YOU give it the `review_dayofweek` column.

Step 2 — Review text length

- **WHAT:** Make `review_length` = character count of review.
- **HINTS:**
 - `df['review_length'] = df['review'].str.len()`.
- **EXPECTED:** A new integer column. Most rows in the range 20-1000.

Step 3 — Capital words and caps ratio

These features measure SHOUTING.

New column	What it is
<code>n_capital_words</code>	How many words are ALL CAPS (like “AMAZING”)
<code>caps_ratio</code>	Of all the letters, what % are uppercase

- **HINTS for `n_capital_words`:**
 - Skeleton:

```
def count_caps_words(text):  
    if not isinstance(text, str):  
        return 0  
    words = text.____() # split into words  
    return sum(1 for w in words if w.____() and len(w) > 1)  
df['n_capital_words'] = df['review'].apply(____)
```
 - Fill in the blanks. (split, isupper, the function name.)
- **HINTS for `caps_ratio`:**
 - Total upper count: `sum(1 for ch in text if ch.isupper())`.
 - Total letter count: `sum(1 for ch in text if ch.isalpha())`.
 - Divide. Watch for division by zero (return 0 if no letters).
- **WHY:** A review that SHOUTS in CAPITAL LETTERS is usually emotional. Could be a happy fan or an angry hater. The model can learn the pattern.

Step 4 — Punctuation count

- **WHAT:** Count special characters: !, ?, ..
- **HINTS:**
 - `df['n_punctuation'] = df['review'].str.count('[!?.]')`.
- **WHY:** Many exclamation marks (!!!) = strong emotion. The model can learn this.

Step 5 — Emoji / non-ASCII count

Gamers often type ASCII art or symbols. We count any character that is NOT a normal letter or number.

- **WHAT:** `n_emoji_codes` = number of characters in the review that are outside basic ASCII.
- **HINTS:**
 - Skeleton:

```
def count_non_ascii(text):  
    if not isinstance(text, str):  
        return 0  
    return sum(1 for ch in text if ord(ch) > __)
```
 - `df['n_emoji_codes'] = df['review'].apply(count_non_ascii)`
 - Fill in the blank with 127 (the last basic ASCII code).
- **WHY:** Steam reviews often use Unicode symbols, ASCII art, emoji. This can signal a meme review (low value) or a real opinion.

Step 6 — Simple sentiment score (rule-based)

- **WHAT:** A simple number that says how positive or negative the review feels.
- **HINTS:**
 - Use the library `textblob`. It is already in Colab.
 - Skeleton:

```
from textblob import TextBlob  
def sentiment(text):  
    if not isinstance(text, str):  
        return 0.0  
    return TextBlob(text).sentiment.polarity
```
 - `df['sentiment_score'] = df['review'].apply(sentiment)`
 - The result is between -1.0 (very negative) and +1.0 (very positive).
- **WHY:** A very negative sentiment score should match `recommended == 0` most of the time. We can check this in Class 4.
- **WARNING:** Sentiment is slow. On 100,000 rows it can take 2-5 minutes. Be patient. Do not stop the cell.

Step 7 — Helpfulness ratio

- **WHAT:** `helpful_ratio = votes_up / (votes_up + votes_funny + 1)`.
- **HINTS:**
 - Skeleton:

```
df['helpful_ratio'] = df['votes_up'] / (df['votes_up'] + df['votes_funny'] + 1)
```
 - The + 1 is to avoid division by zero.
- **WHY:** This separates serious useful reviews (high `helpful_ratio`) from joke reviews (high `votes_funny`).

Step 8 — Game-level features (target-encoded `app_id`)

- **WHAT:** Replace `app_id` with the average recommend rate for that game.
- **HINTS:**
 - `game_rates = df.groupby('app_id')['recommended'].mean()`.
 - `df['game_recommend_rate'] = df['app_id'].map(game_rates)`.
- **WARNING:** This is LEAKAGE if you compute from all data including the row itself. In Module 4 we will fix this with proper cross-validation. For now, write a note in markdown saying “this is leakage, will fix later”.

- **WHY:** Some games are loved by 95% of players. Some are hated. Knowing the average for the game is a strong signal.

Step 9 — Bot/spam flag (anomaly angle)

Erik mentioned: very short review + 0 hours played + still “recommended” = probably a bot.

- **WHAT:** Make a 0/1 column `is_suspicious_bot`.
- **HINTS:**
 - Conditions: `review_length < 20 AND hours_played < 0.1 AND recommended == 1`.
 - Skeleton:

```
df['is_suspicious_bot'] = ((df['review_length'] < __) &
                          (df['hours_played'] < __) &
                          (df['recommended'] == __)).astype(int)
```
- **EXPECTED:** A small percentage (1-3%) of rows flagged.
- **WHY:** This helps us SEE the bot problem. In Module 5 we will study these anomalies more.

Step 10 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/steam_lab/steam_step3.parquet')`.

Phase C — Make ONE chart for Erik

Erik’s chart — “Review length distribution”

A polished histogram of `review_length`.

- **HINTS:**
 - `plt.hist(df['review_length'], bins=100)`.
 - Use `plt.xlim(0, 2000)` to zoom in.
 - Add a vertical line at the median.
- **Title:** “Steam reviews vary wildly in length - median is 80 characters, but 5% of reviews are over 1000 characters.”
- **X-label:** Review length (characters).
- **Y-label:** Number of reviews.
- **Takeaway for Erik:** “The smart critical reviews are likely in the long tail (500+ characters). These are the reviews we want to surface on game pages.”

Common mistakes in Class 3

Mistake	What goes wrong
Apply TextBlob without checking for NaN	Crash on non-string rows.
Compute <code>game_recommend_rate</code> from ALL data without warning	Leakage. Will look amazing in training, fail in production.
Forget +1 in <code>helpful_ratio</code>	Divide by zero error.
Make <code>caps_ratio</code> divide by zero	Crash on reviews with no letters (only symbols).
Use <code>np.log</code> instead of <code>np.log1p</code> on <code>review_length</code>	Crash on length 0.

Self-check before Class 4

- ☐ `review_year`, `review_month`, `review_dayofweek` exist.
- ☐ `review_length`, `n_capital_words`, `caps_ratio`, `n_punctuation` exist.
- ☐ `n_emoji_codes` exists.

- ☐ sentiment_score exists (between -1 and +1).
 - ☐ helpful_ratio exists (between 0 and 1).
 - ☐ 3+ exploratory charts.
 - ☐ 1 explanatory chart.
 - ☐ steam_step3.parquet saved to Drive.
-

Class 4 — Feature Selection

Scenario reminder: Now you have ~25 columns. Erik says: “Too many. Some columns say the same thing. Some are useless. Pick the 12-15 BEST. The model trains faster, the answers are clearer.”

Your goal

Pick the best 12-15 columns. Drop the rest. Justify every choice with numbers.

Inputs

- steam_step3.parquet

Outputs

- steam_step4.parquet in Drive (only the selected columns + recommended)
 - 3+ exploratory charts (correlation heatmap, mutual info bars, importance bars)
 - A short markdown report explaining your choices
-

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap of numeric columns

- **Question:** “Which columns say the same thing as each other?”
- **HINTS:**
 - Pick the numeric columns: `numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()`
 - Compute `df[numeric_cols].corr()`.
 - Plot with `sns.heatmap()`.
 - Use `annot=True` and `fmt='.2f'`.
- **What you learn:** hours_played and hours_played_log will be 0.95+ correlated (we keep the log). votes_up_log and weighted_vote_score may be highly correlated. Pairs of columns with `lccorr > 0.9` are redundant.

Exploratory chart 2 — Mutual information bar chart

- After you compute mutual info (Step 4 below), plot it.
- **HINTS:**
 - Sort the values, then bar chart.
- **What you learn:** Which columns predict recommended strongest. Probably sentiment_score, review_length, hours_played_log, game_recommend_rate.

Exploratory chart 3 — Random Forest feature importance

- After you train the RF (Step 5 below), plot the importances.
- **HINTS:** `pd.Series(rf.feature_importances_, index=...).sort_values().plot.barh()`.
- **What you learn:** A second opinion on feature importance.

Exploratory chart 4 — Hours played vs recommend rate (Erik's chart for THIS class)

- We made it in Class 2 but with raw hours. Make it again with `hours_played_log` and 8 buckets.
 - **HINTS:** `pd.qcut(df['hours_played_log'], q=8)` then group and mean.
 - **What you learn:** Smooth, clean version of the chart that Erik can use in his next leadership meeting.
-

Phase B — Select features

Step 1 — Split into train and test FIRST

- **WHAT:** Use `train_test_split`.
- **HINTS:**
 - `from sklearn.model_selection import train_test_split.`
 - `X = df.drop('recommended', axis=1); y = df['recommended'].`
 - Pass: `test_size=0.2, random_state=42, stratify=y.`
- **WHY:** All next steps use TRAIN ONLY. Otherwise leakage.

Step 2 — Drop the text column FOR THIS STEP ONLY

- **WHAT:** Mutual info and RF do not eat text. Drop review for the selection step.
- **HINTS:**
 - `X_train_num = X_train.drop(columns=['review', 'app_name', 'review_date'])` — drop strings and dates.
 - Keep a copy of the full data; we will add review back in the final file.
- **WHY:** We keep review for Module 7 (text mining). It is not a numeric feature for the predict model in Class 5. Module 4 will add TF-IDF on review.

Step 3 — Remove low-variance columns

- **WHAT:** Drop columns where almost all values are the same.
- **HINTS:**
 - `from sklearn.feature_selection import VarianceThreshold.`
 - `VarianceThreshold(threshold=0.01).fit_transform(X_train_num).`
- **EXPECTED:** Maybe `is_suspicious_bot` is dropped (very few 1s).

Step 4 — Remove highly correlated columns

- **WHAT:** Drop one of each pair where `lccorr > 0.9`.
- **HINTS:**
 - Compute correlation matrix.
 - Get the upper triangle with `np.triu(...)`.
 - For each column, if any of its upper-triangle correlations is `> 0.9`, mark it for dropping.
- **EXPECTED:** Probably drop `hours_played` (we have `hours_played_log`). Maybe drop one of `votes_up_log` or `weighted_vote_score`.

Step 5 — Rank by mutual information

- **WHAT:** Score each column by how much it tells you about recommended.
- **HINTS:**
 - `from sklearn.feature_selection import mutual_info_classif.`
 - `mi = mutual_info_classif(X_train_num, y_train).`
 - Put in a Series, sort.
- **EXPECTED:** Top features: `sentiment_score`, `game_recommend_rate`, `hours_played_log`, `review_length`, `helpful_ratio`.

Step 6 — Random Forest importance (second opinion)

- **WHAT:** Train a small RF, look at `feature_importances_`.
- **HINTS:**
 - `from sklearn.ensemble import RandomForestClassifier.`
 - `RandomForestClassifier(n_estimators=50, max_depth=8, n_jobs=-1, random_state=42).`
 - `.fit(X_train_num, y_train).`
 - Look at `.feature_importances_`.

Step 7 — Pick the final 12-15 columns

- **WHAT:** Combine the rankings. Pick columns that:
 - Are high in mutual info, AND
 - Are high in RF importance, AND
 - Were not dropped by variance/correlation pruning.
- **WRITE DOWN:** In your notebook, list the columns. Explain in 1 sentence why each one is in.

Step 8 — Save

- **HINTS:** Keep only the selected columns + recommended + review (for Module 7) + review_id (for tracking). Save as `steam_step4.parquet` to Drive.

Phase C — Make ONE chart for Erik

Erik's chart — “These are the 10 most important columns”

A horizontal bar chart of your top 10 features and their importance score.

- **HINTS:**
 - Use the RF importance.
 - `sort_values()` then `.head(10)` then `.plot.barh()`.
- **Title:** “Top 10 predictors of recommended - sentiment and game average together predict 60% of the answer.”
- **Y-axis:** column names.
- **X-axis:** Random Forest importance.
- **Takeaway:** “Three things matter: how the player feels (sentiment), what the game average is (game_recommend_rate), and how much the player played (hours_played_log). Everything else is small.”

Common mistakes in Class 4

Mistake	What goes wrong
Compute mutual info on the FULL data	Leakage.
Drop columns without writing why	Module 4 students will not understand.
Keep too many columns (40+)	Slow training. Overfitting risk.
Drop a high-mutual-info column	Big mistake. Check why it is high before dropping.
Drop the review text column from the final file	We need it for Module 7. Keep it.
Forget <code>stratify=y</code> in train/test split	Train and test get different recommend rates. Bad.

Self-check before Class 5

- ☐ Train/test split done FIRST.
- ☐ 12-15 numeric columns remain + recommended + review + review_id.
- ☐ You wrote down WHY for each kept column.

- ☐ 3+ exploratory charts.
 - ☐ 1 chart for Erik.
 - ☐ steam_step4.parquet saved to Drive.
-

Class 5 — Pipelines

Scenario reminder: Erik says: “Your cleaning code is in 4 different notebooks. When a new review arrives tomorrow, the model needs to clean it the same way. You cannot copy 4 notebooks to a server. We need ONE object that does everything.”

Your goal

Put EVERY cleaning step inside ONE Pipeline object. Production-ready code.

Inputs

- steam_step4.parquet (selected columns)

Outputs

- steam_pipeline.joblib saved in Drive
 - 1 confusion-matrix chart + 1 ROC curve chart
 - A pipeline that takes RAW input and produces predictions
-

Phase A — Explore the data first

Exploratory chart 1 — Confusion matrix (after training)

- After Step 6, build a confusion matrix.
- **HINTS:**
 - from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay.
 - cm = confusion_matrix(y_test, y_pred).
 - ConfusionMatrixDisplay(cm, display_labels=['Not recommended', 'Recommended']).plot().
- **What you learn:** How many critical reviews did we CATCH? How many did we miss?

Exploratory chart 2 — ROC curve

- **HINTS:**
 - from sklearn.metrics import RocCurveDisplay.
 - RocCurveDisplay.from_estimator(pipeline, X_test, y_test).
- **What you learn:** The trade-off between catching critics and false alarms. The AUC number.

Exploratory chart 3 — Hours played vs recommend rate (the recurring Erik chart)

- One more time, after the pipeline is trained. Compare the model's prediction rate to the actual rate by hours bucket.
 - **What you learn:** Does the model agree with the data we saw in Classes 2-4?
-

Phase B — Build the pipeline

Step 1 — Decide which columns are numeric and which are categorical

- **HINTS:** Make two lists.
- **EXAMPLE:**
 - numeric: hours_played_log, review_year, review_month, review_dayofweek, votes_up_log, votes_funny_log, weighted_vote_score, helpful_ratio, review_length, n_capital_words, caps_ratio, n_punctuation, n_emoji_codes, sentiment_score.
 - categorical: maybe none for this model — we already encoded everything. (Or app_id if we keep it.)
- **NOTE:** review (the text) is NOT in either list for THIS class. Module 4 will add a third pipeline branch with TF-IDF for the text.

Step 2 — Build the numeric mini-pipeline

- **WHAT:** 2 steps: imputer (fill missing) + scaler.
- **HINTS:**
 - from sklearn.pipeline import Pipeline.
 - from sklearn.impute import SimpleImputer.
 - from sklearn.preprocessing import StandardScaler.
 - Skeleton:

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('scaler', StandardScaler()),
])
```
 - Fill in the blank: best strategy for numeric? ('median')

Step 3 — Build the categorical mini-pipeline (if needed)

- **WHAT:** If you have categorical columns, 2 steps: imputer + one-hot encoder.
- **HINTS:**
 - from sklearn.preprocessing import OneHotEncoder.
 - Skeleton:

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('onehot', OneHotEncoder(handle_unknown='___', sparse_output=___)),
])
```
 - Fill in blanks. (Strategy 'most_frequent', handle_unknown 'ignore', sparse_output False.)
- **WHY handle_unknown='ignore'?** In production, a new app_id may appear. We do not want the code to crash.

Step 4 — Combine into a ColumnTransformer

- **HINTS:**
 - from sklearn.compose import ColumnTransformer.
 - It takes a list of tuples: (name, transformer, list_of_columns).

Step 5 — Add the model on top

- **HINTS:**
 - from sklearn.linear_model import LogisticRegression.
 - LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42).
 - Put it in a Pipeline with the preprocessor.
- **WHY class_weight='balanced'?** Only 20% of reviews are “not recommended”. Without this, the model just predicts “recommended” for everyone and gets 80% accuracy — but it never catches the critics. Useless for Erik.

Step 6 — Train and evaluate

- **HINTS:**
 - `full_pipeline.fit(X_train, y_train).`
 - `y_pred = full_pipeline.predict(X_test).`
 - `from sklearn.metrics import classification_report.`
 - Read the F1 of the “0” class (not recommended). That is the number Erik cares about.
- **EXPECTED:** F1 on the “not recommended” class around 0.40-0.55. (Module 4 will improve this with TF-IDF on the text.)

Step 7 — Save the trained pipeline

- **HINTS:**
 - `import joblib.`
 - `joblib.dump(full_pipeline, '/content/drive/MyDrive/steam_lab/steam_pipeline.joblib').`

Phase C — Make ONE chart for Erik

Erik’s chart — “How many critical reviews did we catch?”

A simple confusion matrix with clear labels:

	Predicted recommended	Predicted not recommended
Actually recommended	true positives	false alarms
Actually not recommended	missed critics	caught critics

- **HINTS:** Print the confusion matrix with `sns.heatmap()` and `annot=True`.
- **Title:** “Of 4,000 critical reviews in the test set, our baseline model catches ~2,200.”
- **Takeaway for Erik:** “We catch about 55% of critical reviews using only numeric features. Module 4 will add the text (TF-IDF) and we expect this number to go up to 70-80%.”

Common mistakes in Class 5

Mistake	What goes wrong
Build the pipeline AFTER you scaled the data manually	Pipeline scales it twice. Numbers wrong.
Forget <code>sparse_output=False</code> in <code>OneHotEncoder</code>	Some downstream code breaks.
Use <code>class_weight='balanced'</code> AND SMOTE	Over-correction. Pick one.
Forget <code>random_state</code>	Results change every run. Hard to debug.
Try to fit review (text) into the numeric pipeline	Crash. Save text for Module 4.

Self-check before Class 6

- ☐ Pipeline has preprocessor + model.
- ☐ Preprocessor has numeric transformer.
- ☐ `class_weight='balanced'` set.
- ☐ Confusion matrix + ROC curve charts.
- ☐ `steam_pipeline.joblib` saved to Drive.

Class 6 — End-to-End Lab (THE BIG ONE)

Scenario reminder: Erik is in the meeting room. He wants the FINAL clean dataset on his desk in 90 minutes. This is the lab.

Your goal

Take the 1 raw CSV file (6.4M rows). Produce ONE final .parquet file with the exact 19-column schema. Plus a 1-page findings report.

Time

90 minutes of focused work.

Inputs

- The 1 raw CSV file (`dataset.csv`) in your Drive folder

Outputs

- `steam_reviews_clean.parquet` (~100,000 rows x 19 columns) in Drive
 - `findings.md` (~1 page)
 - Your full Colab notebook
-

Phase A — Explore the data first (10 minutes)

You will reuse charts from Classes 1-5. Pick 3 of them. Put them all on ONE PAGE of your notebook with markdown headers: 1. Language distribution before filtering (Class 1) 2. Hours played vs recommend rate (Class 2) 3. Top 10 most important features (Class 4)

These 3 charts tell Erik the whole story.

Phase B — Build the final dataset (70 minutes)

Required output schema

Your `steam_reviews_clean.parquet` MUST have these 19 columns, with these names and types:

#	Column	Type	Source
1	<code>review_id</code>	int	raw (tracking)
2	<code>app_id</code>	int	raw (game id)
3	<code>language</code>	string	raw (will be all “english”)
4	<code>review</code>	string	raw text - kept for Module 7
5	<code>hours_played_log</code>	float	engineered (log of hours played)
6	<code>review_year</code>	int	engineered (from <code>timestamp_created</code>)
7	<code>review_month</code>	int	engineered
8	<code>review_dayofweek</code>	int	engineered (0=Mon, 6=Sun)

#	Column	Type	Source
9	votes_up_log	float	engineered (log of votes_up)
10	votes_funny_log	float	engineered
11	weighted_vote_score	float	raw (Steam's helpfulness score)
12	helpful_ratio	float	engineered (votes_up / (votes_up + votes_funny + 1))
13	review_length	int	engineered (character count)
14	n_capital_words	int	engineered
15	caps_ratio	float	engineered (% of letters that are uppercase)
16	n_punctuation	int	engineered
17	n_emoji_codes	int	engineered (non-ASCII char count)
18	sentiment_score	float	engineered (TextBlob polarity)
19	recommended	int (0/1)	TARGET

Lab phases (timed)

Phase	Time	What you do
1. Load + filter + sample	15 min	Read in chunks. Filter to English. Sample 100,000 with seed 42.
2. Clean	10 min	Convert timestamp_created to date. Convert minutes to hours. Cast booleans to int. Drop missing reviews.
3. Engineer time features	10 min	review_year, review_month, review_dayofweek.
4. Engineer text features	25 min	review_length, n_capital_words, caps_ratio, n_punctuation, n_emoji_codes, sentiment_score. (Sentiment is slow.)
5. Engineer numeric features	10 min	Log-transforms. helpful_ratio.
6. Validate + save	10 min	Check schema. 19 columns. Right dtypes. Save .parquet.
7. Findings	10 min	Write findings.md.

Total: 90 minutes.

NOTE: 90 minutes is tight, especially because sentiment_score is slow on 100,000 rows. Start that cell EARLY. While it runs, work on the other features in parallel cells.

Phase C — Findings report for Erik (10 minutes)

Write findings.md with these sections:

Final numbers

- Final row count: _____
- recommended rate: _____% (should be 75-82%)
- Most common review length bucket: _____
- Median hours played: _____

Top 3 insights

1. _____
2. _____
3. _____

One question to investigate in Module 4

- _____

One chart that summarizes everything

Embed your most important chart (hours played vs recommend rate, or review length distribution).

Grading rubric (100 points)

Item	Points
All 19 columns exist with the right names and dtypes	25
Cleaning decisions written in markdown	10
Pipeline is reproducible (re-run from the raw CSV in ONE command)	15
English filter applied BEFORE sampling	10
random_state=42 used in .sample()	5
recommended rate is between 75-82%	5
hours_played_log exists and looks reasonable (no infinity)	5
At least 3 exploratory + 1 explanatory chart per class	15
findings.md has insights, not just numbers	10

Submit

Upload to module-3/class_6/submissions/<YourTeamName>/: - steam_reviews_clean.parquet - Your Colab notebook (File > Download > Download .ipynb) - findings.md

Tips for the whole module

1. **Do NOT copy-paste code from this guide.** This guide gives you HINTS only. Write the code yourself. You will learn much faster.
2. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
3. **Save intermediate .parquet files to Drive** (steam_step1, steam_step2, etc.). If something breaks, you do not redo everything.
4. **Pair-program.** One student types, one reads. Switch every 20 minutes.

5. **Make a chart BEFORE you write cleaning code.** You must SEE the problem before you can fix it.
6. **Make a chart AFTER each step.** Confirm your code did what you expected.
7. **Ask the mentor early.** If you are stuck for 20 minutes, ask. Do not waste an afternoon.
8. **Filter before sampling.** Always. The order matters: first reduce to English, then take 100,000 rows.
9. **Always use random_state=42.** Reproducibility is a job skill, not just a class rule.
10. **Sentiment analysis is slow.** Start it early. Use it as background while you write other code.

Good luck.